CS 5004 Final Project Instructions

Term Projects encourage students to further explore and practice key ideas from the course through authentic, tangible activities relevant to personal goals and interests. Projects also provide an opportunity for students to demonstrate mastery of material through more compelling, less stressful assignments instead of traditional timed assessments. Humans learn by doing and then reflecting on the results. Creating a project tends to feel more like the career work you hope to do than undergoing a traditional Final Exam.

Several elements are required for a successful project.

A. During Final Exam week, an oral presentation, including slides, a demo, and a brief code review, should occur in front of the class. An audience of people (the instructional team plus your peers) should be present to observe, ask questions, and offer suggested improvements. Learning to present your work in front of an audience is a crucial skill to develop in graduate school; being a little nervous about public speaking is normal until you have done it numerous times. That is OK. Presentations will be scheduled between 1:00 and 4:00 PM on Tuesday, 04/21/2026.

B. Your slides may be created in any commonly used format (PowerPoint, Keynote, or Google Preso are fine; others could also work). Your presentation should include:

1. **Goals** and **Rationale**
2. **Demonstration** of your project in operation
3. **Methods and Tools Used**
4. **Walk-through** of a small portion of your **code**
5. **Findings and Lessons Learned** (What was hard/easy?)
6. **Limitations** and **Future Extensions**
7. **Highlights** of your use of 1-2 **big ideas** from **Object Oriented Design** (*e.g.,* encapsulation, inheritance, composition, polymorphism, abstraction, MVC)
8. **Scholarly Citations** (or at least URLs) for resources you relied upon
9. Time for **Questions** and **Answers**

C. The final commit and push to your Final Project URL on GitHub **must occur by 12:00 PM (PT) on Tuesday, 04/21/2026**. **<u>Late work</u> or <u>missing files</u> cannot be accepted due to the non-negotiable** Registrar grade submission deadline. Turn in whatever you have by then; every project has room for improvement.

D. Your **choice of topic** is flexible. We understand that your time for this assignment is finite. Details may evolve somewhat as your work progresses, but **you must alert us to significant changes!** Learning to define a topic that balances challenge and interest versus realism is part of the joy and benefit of graduate school.

E. Your topic and any changes must be pre-approved by the Instructor because it is common for students to propose unrealistically ambitious projects or projects that do not adequately

highlight the critical ideas of CS5004. The attached Appendix suggests some ideas to help with your brainstorming. Please do not feel obligated to choose ***any*** of these – they are just examples to help you get started! Other topic ideas may be shared in class.

F. **Team projects** are encouraged within the following constraints: The maximum team size is **five** students. Solo projects are permitted in rare cases if the students are full-time employees and are not able to meet other teammate at campus. Please divide the project into multiple parts for multiple team members. **All participants should have some involvement in each of the major elements of the project**; for example, one person doing the entire "front end" and another person doing the entire "back end" is not OK. Workload should be shared evenly within a team.

G. Your topic should require predominantly **Java** programming. **Websites typically require considerable JavaScript and CSS coding, so they may not be ideal for this course**. **Similarly, iPhone apps typically require extensive coding in Apple's Swift language, so they may not be the best choice for CS5004**.

APPENDIX A

A Few Sample Project Ideas, Just to Help Catalyze Brainstorming

A. Here are a few sample project ideas just to help with brainstorming.

- A Java program to play a game. This could be a game for two humans, with the computer's job being to provide suitable graphics and record-keeping. Games can be an excellent opportunity to demonstrate mastery of MVC or using a GUI such as Swing.
- A reimplementation of your solution to a previous CS5004 assignment, but running on a website, including a description of what was involved in making that happen. However, **most of your project should focus on OOD and be coded in Java**. Be careful here.
- A Text Adventure Game (such as https://legacy.learningtech.org/adventure/, in Java).
- A *rule-based* Java program that checks a text file of Java code to alert students to poor choices of variable names or other guidelines not typically flagged by IntelliJ.
- A password checker that prompts for a password and verifies that it is strong enough.
- A TicTacToe game using the minimax algorithm *or* heuristic programming for an "AI" player (even with a simple textual user interface).
- A Java Program to generate semi-random poetry, music, or artistic patterns.
- A Java Program to exercise an image processing library or a GUI library for Java.
- A graphics-oriented interactive game, such as a variant of Pong.
- A desktop app to advise children on how to care for certain types of pets.
- A Java "reminder app" that interacts with users via Discord or Slack.
- A *web scraper* for tickets, stock prices, movie recommendations, etc. Be careful here, though, because many websites are deploying techniques to cause scrapers to fail.
- A Personal Health/Fitness Assistant.
- An airport Flight Information System (demo scale).
- A Music Player, perhaps with setlist editing capabilities.
- A Chess Game Manager or Chess Puzzles such as *Knights Tour* or *Eight Queens*.
- An Employee Management System or variations on this idea.
- A Parking Spot Locator.
- An Apartment Locator.
- An Online Shopping tool, perhaps for food ingredients for favorite meals

APPENDIX B: Overall Scoring Rubric

ITEM	POSSIBLE SCORE	SUBTOTALS
Submitted (Presented & Submitted on Schedule)	23	23
Content		
Complexity (Balanced level of ambition)	7	27
Creativity/Originality	4	
Can we build and run the code?	3	
Relevance to CS5004 tools	9	
Resources and Citations	4	
Code Quality/Style Rubric		
Clearly Written, Javadocs and inline comments	4	16
Broken into appropriately sized sub-problems	4	
Appropriate data structures and algorithms	4	
Code is testable and tested with Adequate Coverage	4	
Object Oriented Design Principles		
Follows 22 design principles (check inputs, exceptions, catch/handling errors, public vs private), style, naming	22	22
Other		
Professionalism of Presentation	2	12
Quality, Clarity, Readability of Slides	2	
Timing (approx. 15 mins, depending on team size)	2	
Lessons Learned	2	
Demo	2	
Explanation of Code	2	
Grand Total		100

Explanations for some of the terms shown above follow.

- **Relevance is to the “big ideas” such as OOP and skills (to use the available resources from the course, for example use of Swing for GUI instead of other frontend tools)** taught in CS5004-5005.
- Creativity or originality. It is OK to create a novel solution to an already-solved problem. However, please do not follow a "YouTube Cookbook" type solution! Projects must reflect primarily your own work.
- A balanced level of ambition means challenging yet realistically doable.
- Completion of Working Code for a reasonable subset of your project goal.
- Your coding style should be modular, follow Java naming conventions, format comments correctly, and comply with OOD style and design guidelines.
- Quality and clarity of slides (and any other written materials) matters. Slides should use large, readable fonts with high contrast (black text on a white background).
- Scholarly citations for all information sources (minimally, URLs) are expected. All materials submitted and supporting resources must be readily available in English.
- Ability to answer detailed questions about your ideas and program operation matters.
- Group project contributors must each be involved in nearly all aspects (e.g., coding, design, presentation).
- Further details about the code quality rubric follow.

APPENDIX C: The Mullen Coding Rubric

Best practices for Design/Coding (incorporated into the overall rubric)

	Project Requirements balance challenge with realism	**Code is clearly written and well commented / documented** *(Emphasis on Class structure and correct Javadocs for CS5004)*	**Problem is broken into appropriately sized sub-problems. Elements contain appropriate amounts of functionality**	**Data types, data structures, and algorithms are used appropriately**	**Code is written to be testable. Test coverage is complete**
Exemplary	Project must be sufficiently complex that code demonstrates thoughtful design choices including structuring for testability. Solutions should include multiple testable methods.	Code is clear and well documented	All interfaces, classes, and methods contain appropriate levels of functionality (each class has clear responsibility, each method does 1 job)	Data types are well-chosen and match requirements of the data	Code test coverage is complete, and tests are easy to write and read. All tests pass
Proficient	Project involves the use of ADTs or data structures in a way that requires students to choose which of several options are best for task, or involves multiple implementations of an interface	Most code is clear, most expected documentation is in place	Most functions or methods contain appropriate amounts of functionality	Data types are mostly appropriate.	Code test coverage is complete but not all tests are successful
Basic	Project must be sufficiently complex to require multiple classes and/or methods, and to require students to make organizational choices in how their code is structured	Documentation missing and/or significant portions of code are unclear	More than one function/method exist but the division of functionality is confusing	Data may be mis-classified or coerced in inappropriate ways, but the code still runs	Portions of the code are tested, but not all tests pass or tests are not written to appropriately test code functionality
Novice	Project must include multiple classes and methods written from scratch by the student	Code is unclear and/or undocumented	All code is in one class, method, or function	Data types are incompatible with data; code may throw errors related to data types	Code is inadequately tested and is structured in a way that makes testing difficult

APPENDIX D: Design Principles

1) Javadoc all public classes and methods. Class comments should be at least two sentences long and provide information not already clear from their definition.
2) Use interface types over concrete classes wherever possible. Exception: immutable "value" objects. Classes with no interface.
3) Fields must always be private. Exception: constants. Methods and classes should be as private as possible.
4) Class should never have public methods not in the interface (aside from constructor).
5) Catch and handle/report errors as early as possible.
6) Check inputs.
7) Use exceptions only for exceptional situations – not for flow control.
8) Beware of references, copies, and mutation. Make defensive copies.
9) Separate responsibilities: one class, one responsibility.
10) Do not duplicate code.
11) Open for extension, closed for modification: make changes without modifying existing code; write code to support later changes without modification.
12) Extensibility: design to make likely later changes easier.
13) Write tests first, cover the range of situations, edge cases. Write code to be testable (e.g., avoid System.out); do not expose fields or add public methods just for testing.
14) Loose coupling over tight coupling. Write reusable components when possible.
15) You can't change an interface once it's published.
16) Reuse existing exceptions, classes, libraries, and design